SMART HOME HUMAN MACHINE INTERFACE

ESP32 NodeMCU-32S Residential Automation Controller

PROJECT DOCUMENTATION

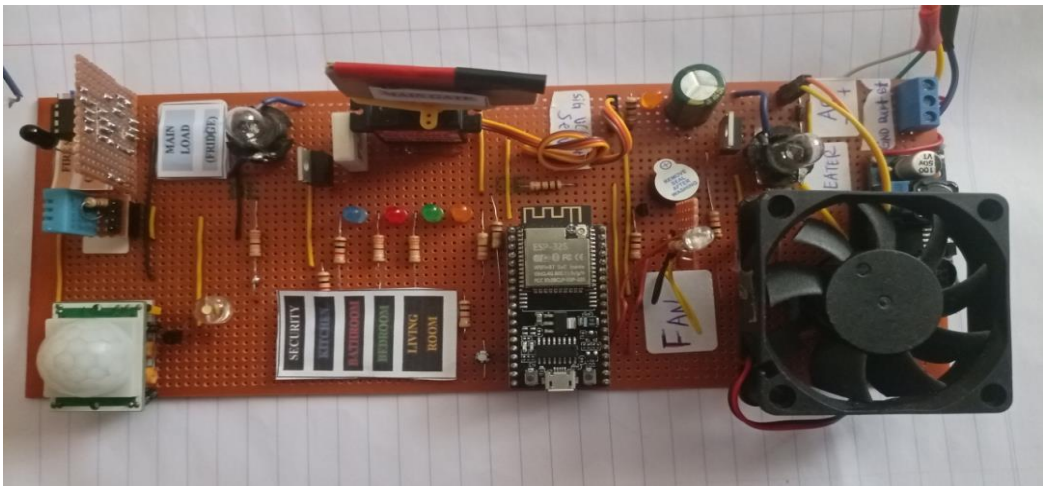

Figure 1: Completed G5 Smart Home HMI Hardware Build

Attached Project Files

File	Description
main.py	MicroPython firmware — WiFi server, sensor polling, hardware control, PIN authentication
index.html	Dashboard front-end — served directly from ESP32 flash over HTTP
hmi_pin.txt	Auto-generated PIN configuration file (persisted to ESP32 flash, survives reboot)

May 2026

Abraham Winston Ogoch

Linked in Profile: linkedin.com/in/abraham-winston-ogoch-65b651284

Table of Contents

Table of Contents 2
1. Project Overview 4
1.1 Key Features 4
1.2 System Architecture 5
2. Components and Materials 6
2.1 Electronic Components 6
2.2 Software and Tools 7
3. Pin Configuration and Connections 8
3.1 Power Supply Architecture 9
3.2 Voltage Divider for ADC (GPIO33) 9
3.3 Current Sensing (GPIO34) 9
4. Software Architecture and Description 10
4.1 main.py — Firmware 10
4.1.1 Hardware Initialisation 10
4.1.2 Shared State Dictionary 10
4.1.3 PIN Authentication System 10
4.1.4 Sensor Polling (poll_sensors) 10
4.1.5 Fire State Machine (poll_fire) 11
4.1.6 HTTP Request Handler 11
4.1.7 Main Loop 11
4.2 index.html — Dashboard Front-End 11
4.2.1 PIN Gate Overlay 12
4.2.2 Dashboard Tab 12
4.2.3 System Security Tab 12
4.3 hmi_pin.txt — PIN Configuration File 12
5. Uploading Code to the ESP32 13
5.1 ESP32 NodeMCU-32S Reference 13
5.2 Installing MicroPython Firmware 15
5.3 Uploading Project Files with Thonny 16
6. System Operation and Working 17
6.1 Boot Sequence 17
6.2 Environmental Monitoring 17
6.3 HVAC Automation 17
6.4 Security and Motion Detection 17
6.5 Fire Detection 18

6.6 Power Measurement 18
6.7 Gate Control 18
7. Challenges Faced and Solutions 19
7.1 HTTP Server — Truncated Requests (Bug 1) 19
7.2 HTTP Server — Browser Timeout on Second Request (Bug 2) 19
7.3 DHT11 Sensor Read Failures 19
7.4 ESP32 ADC Non-Linearity 20
7.5 Power Bank / Battery Auto-Shutoff 20
7.6 MicroPython Heap Memory 20
8. GitHub README Summary 21
Appendix 23
Appendix A: Circuit Diagram 23
Appendix B: Firmware Configuration Constants 24
Appendix C: API Endpoint Reference 25
Appendix D: Troubleshooting Guide 25

1. Project Overview

The G5 Smart Home HMI is a self-contained residential automation controller built on the ESP32 NodeMCU-32S microcontroller running MicroPython firmware. It integrates environmental sensing, security monitoring, fire detection, HVAC automation, power measurement, and remote control through a browser-based dashboard accessible from any device on the local WiFi network.

The system hosts its own web server directly on the ESP32, serving a full-featured single-page dashboard (index.html) from the device's internal flash memory. No cloud connectivity, external server, or internet access is required. The entire user interface is accessible by navigating to the ESP32's local IP address in any web browser — on a phone, tablet, or desktop.

A 4-digit PIN security system is implemented entirely in firmware, persisting across reboots via a plain-text configuration file on the ESP32 flash. When PIN protection is enabled, any browser visiting the IP address is presented with a PIN entry keypad before gaining access to the dashboard.

1.1 Key Features

- Real-time environmental monitoring: temperature (°C and °F) and relative humidity via DHT11 sensor
- PIR-based motion detection with audible buzzer alarm and automatic re-arming after 10 seconds
- Fire detection sensor with a three-stage safety state machine: Fire → Putting Out → Cooldown → Safe
- RGB LED fire status indicators (Red = fire, Blue = cooldown, Green = safe)
- Servo-controlled gate (open/close) with position feedback on the dashboard
- Individual room lighting control for four rooms: Bedroom, Bathroom, Kitchen, Living Room
- Automatic HVAC control: heater activates below a user-set threshold; fan activates above another threshold
- Main load relay (fridge/appliance) with manual toggle via the HMI
- White security light toggle (GPIO22)

- Power metering via ADC: voltage, current, real power, cumulative energy (Wh), and estimated cost in KSH
- 4-digit PIN access control: set, change, enable/disable, or remove via the System Security tab
- PIN persists to flash (hmi_pin.txt) and survives power cycles
- Responsive, colour-coded dashboard with animated status indicators and fire overlay

1.2 System Architecture

The ESP32 runs a single-threaded MicroPython event loop. A background poll runs every 1,000 ms to read all sensors, run the fire state machine, and compute power/energy accumulators. The HTTP server uses non-blocking socket accept with a 50 ms timeout, draining up to four queued connections per loop tick to avoid browser request timeouts. The front-end polls /state every 1,500 ms and issues /cmd requests for user actions, with the session cookie (g5t) validated on every gated request when PIN protection is active.

2. Components and Materials

2.1 Electronic Components

#	Component	Qty	Spec / Value	Purpose
1	ESP32 NodeMCU-32S Development Board	1	240 MHz dual-core	Main microcontroller and WiFi web server
2	DHT11 Temperature & Humidity Sensor	1	±2°C, ±5% RH	Ambient temperature and humidity measurement
3	PIR Motion Sensor (HC-SR501)	1	3–5V, 7m range	Motion detection for security alarm
4	Passive Buzzer	1	5V, 85 dB	Audible alarm on motion detection
5	Servo Motor (SG90 or MG90S)	1	5V, 0–180°	Gate open/close (0° = closed, 90° = open)
6	5V Single-Channel Relay Module	1	10A/250VAC	Switches main load (fridge / 12V incandescent)
7	Fire / Flame Sensor Module	1	IR, digital out	Fire detection (HIGH = fire)
8	LED — Red	1	5mm, 20 mA	Fire active indicator (GPIO2)
9	LED — Green	1	5mm, 20 mA	Safe / no fire indicator (GPIO15)
10	LED — Blue	1	5mm, 20 mA	Post-fire cooldown indicator (GPIO21)
11	LED — White (High-brightness)	1	10mm, 20 mA	Manual security light (GPIO22)
12	LED — Orange/Yellow (Room indicators)	4	5mm, 20 mA	Bedroom, Bathroom, Kitchen, Living Room
13	12V Incandescent Bulb (Heater substitute)	1	12V, 300 mA	Represents heater load — GPIO23 via relay/transistor
14	12V Incandescent Bulb (Main Load)	1	12V, 300 mA	Represents main load (fridge) — GPIO4 relay
15	DC Fan (5V or 12V)	1	12V brushless	Represents cooling fan — GPIO5 via transistor
16	Voltage Divider (10kΩ / 1kΩ)	1	Resistors	Scales 14V supply down to 0–3.3V for ADC GPIO33
17	Current Sense Resistor (Shunt)	1	0.22 Ω, 1W	Current measurement via ADC GPIO34 (0–1.1V range)

18	NPN Transistors (e.g. 2N2222 / BC547)	3	≥500 mA	Fan, heater bulb, and auxiliary load switching from 3.3V GPIO
19	Resistors (220Ω for LEDs, 1kΩ base)	~10	1/4W	Current limiting for LEDs and transistor bases
20	Electrolytic Capacitors (100µF, 10µF)	2–3	25V rated	Power supply decoupling and noise filtering
21	14V DC Battery	1	14V nominal	Primary power supply for the full system
22	5V DC-DC Buck Converter / Regulator (LM2596)	1	≥1A output	Steps 14V down to 5V for ESP32 VIN, PIR sensor and servo
23	Perfboard / Veroboard	1	~20×10 cm	Main construction substrate for all components
24	Jumper Wires / Hookup Wire	Assorted	22–26 AWG	Point-to-point wiring on perfboard
25	USB Micro-B Cable	1	Data capable	Firmware upload from PC to ESP32 only (also for powering ESP32 via external power supply or power bank)

2.2 Software and Tools

Software	Version / Type	Purpose
Thonny IDE	v4.x (latest)	MicroPython IDE for editing and uploading code to ESP32
MicroPython Firmware	v1.22+ for ESP32	Python runtime flashed to the ESP32
esptool.py	v4.x	Command-line tool for flashing MicroPython firmware
Python 3	v3.8+	Required by esptool on the host PC
Web Browser	Any modern browser	Accessing the HMI dashboard at the ESP32 IP address
USB-Serial Driver (CP210x or CH340)	OS-dependent	Required on host PC to communicate with ESP32 over USB

3. Pin Configuration and Connections

The ESP32 NodeMCU-32S operates at 3.3V logic on all GPIO pins. All external loads drawing more than approximately 12 mA must be driven through a transistor or relay, not directly from the GPIO pin. The following table lists every pin used in this project, its direction, the connected peripheral, and any necessary interface circuitry.

GPIO	Dir.	Peripheral	Signal	Notes
GPIO2	OUT	LED Red	Fire indicator	220Ω series resistor to GND. Active HIGH = fire.
GPIO4	OUT	Relay	Main load	5V relay module. GPIO4 HIGH energizes coil; NC/NO routes 14V to load bulb.
GPIO5	OUT	Fan	Fan PWM/SW	NPN transistor (1kΩ base resistor). GPIO5 HIGH = fan runs. 12V fan on collector.
GPIO13	IN	PIR HC-SR501	Motion OUT	Internal pull-down. HIGH = motion detected. Sensor VCC to 5V.
GPIO14	IN	DHT11	Data	4.7kΩ pull-up to 3.3V on data line. VCC to 3.3V.
GPIO15	OUT	LED Green	Safe indicator	220Ω series resistor to GND. Lit at boot (safe state).
GPIO18	OUT	Servo Motor	PWM (50 Hz)	5V servo signal. 26–123 duty count maps 0–180°. 0° closed, 90° open.
GPIO19	OUT	Buzzer	Alarm drive	Passive buzzer to GND. HIGH = sound. Optionally add 100Ω series resistor.
GPIO21	OUT	LED Blue	Cooldown indicator	220Ω series resistor to GND.
GPIO22	OUT	LED White	Security light	220Ω series resistor to GND. Manual toggle from HMI.
GPIO23	OUT	Heater bulb	Heater switch	NPN transistor (1kΩ base). GPIO23 HIGH = heater (12V bulb) ON.
GPIO25	OUT	LED (Bedroom)	Room light	220Ω series resistor to GND.
GPIO26	OUT	LED (Bathroom)	Room light	220Ω series resistor to GND.
GPIO27	OUT	LED (Kitchen)	Room light	220Ω series resistor to GND.

GPIO32	OUT	LED (Living Rm.)	Room light	220Ω series resistor to GND.
GPIO33	IN	ADC Voltage	V sense (11dB)	10kΩ/1kΩ divider scales 14V to ~1.27V max. ATTN_11DB = 0–3.6V range.
GPIO34	IN	ADC Current	I sense (0dB)	0.22Ω shunt. Max 5A = 1.1V drop. ATTN_0DB = 0–1.1V range. Input-only pin.
GPIO35	IN	Fire sensor	Digital fire	HIGH = fire detected. Input-only pin. Sensor VCC to 3.3V or 5V per module spec.

3.1 Power Supply Architecture

The system is powered from a 14V DC battery. A DC-DC buck converter steps this down to 5V for the ESP32 (via VIN pin), the servo motor, PIR sensor and relay coil. The ESP32's internal LDO regulator further steps 5V down to 3.3V for all GPIO peripherals and sensors. The 12V loads (fan, heater bulb, main load bulb) are driven directly from the battery rail through NPN transistors or a relay, switched by GPIO outputs.

> IMPORTANT: Never connect 14V or 5V directly to an ESP32 GPIO pin. All loads above 12 mA must be buffered through a transistor (NPN for low-side switching) or relay module. The ESP32 GPIO sinks/sources a maximum of 12 mA per pin and 40 mA total.

3.2 Voltage Divider for ADC (GPIO33)

The 14V battery voltage is measured using a resistive divider. With R1 = 10kΩ and R2 = 1kΩ, the divider ratio is 1/11, so 14V at the input produces approximately 1.27V at the ADC pin. The firmware reads the ADC value (0–4095 for 12-bit), scales it by (3.6 / 4095), then multiplies by 11 to recover the original voltage. ATTN_11DB is set for a 0–3.6V input range.

3.3 Current Sensing (GPIO34)

A 0.22Ω shunt resistor is placed in series with the main load. At 5A the voltage across the shunt is 1.1V. The ADC is configured with ATTN_0DB for a 0–1.1V input range. The firmware divides the measured shunt voltage by 0.22 to recover current in amperes. GPIO34 is an input-only pin on the ESP32 and has no internal pull-up or pull-down.

4. Software Architecture and Description

The project consists of three files that must be present on the ESP32 flash root (/). MicroPython automatically executes main.py on boot.

4.1 main.py — Firmware

main.py is the sole server-side program. It is structured into the following sections:

4.1.1 Hardware Initialisation

All GPIO pins are configured at module level before any function runs. The DHT11 data object, ADC channels, PWM servo, and relay are all instantiated here. The green LED is set HIGH at boot to indicate the safe/no-fire state.

4.1.2 Shared State Dictionary

A single Python dictionary (state) holds all sensor readings and device states. This is the single source of truth for the HMI. It is serialized to JSON and returned verbatim in every /state and /cmd response. The front-end polls this JSON every 1,500 ms to refresh all dashboard widgets.

4.1.3 PIN Authentication System

The PIN is stored in hmi_pin.txt on the ESP32 flash in the format pin,enabled — for example 1234,1 or ,0 (no PIN set, disabled). On boot, _pin_load() reads this file. The firmware issues a random 16-byte hex session token (g5t cookie) on successful PIN login via /pin/login?p=XXXX. All gated routes (/state, /cmd, /, /pin/save) validate the cookie before serving. /pin/status and /pin/login are always reachable without authentication so the browser can show the PIN overlay.

4.1.4 Sensor Polling (poll_sensors)

Called every 1,000 ms from the main loop and also inline on every /state or /cmd request for sub-second responsiveness. It reads DHT11 (temperature, humidity), PIR motion, buzzer timer, fire sensor state machine, ADC voltage and current, computes power and cumulative energy, and updates the state dictionary.

4.1.5 Fire State Machine (poll_fire)

The fire sensor output is polled on every sensor cycle. The machine has four states:

- safe — green LED on, no fire. This is the idle/boot state.
- fire — red LED on, green off. Entered immediately when fire sensor reads HIGH. A 5-second timer starts when the sensor first drops LOW.
- putoff — blue LED on. Entered 5 seconds after fire sensor drops LOW. A 20-second blue-LED hold timer starts.
- cooldown — transitional state lasting one poll cycle to turn off the blue LED and restore the green LED before returning to safe.

4.1.6 HTTP Request Handler

_recv_request() reads the TCP stream until the HTTP header terminator \r\n\r\n is received, capped at 4 KB. This prevents the original bug where the loop broke prematurely on the first sub-512-byte chunk. handle_request() parses the first line of the request to extract the method, path, and query string, then dispatches to the appropriate handler block.

4.1.7 Main Loop

The main loop runs with srv.settimeout(0.05) — a 50 ms socket accept timeout. Each iteration checks whether 1,000 ms has passed for sensor polling, then drains up to four queued TCP connections before the next iteration. This prevents the bug where a browser's second request (fired immediately after receiving the HTML) would time out because the server was busy handling the first.

4.2 index.html — Dashboard Front-End

index.html is a self-contained single-page application served from ESP32 flash. It is approximately 49 KB and uses no external JavaScript frameworks. All JavaScript, CSS, and HTML are in one file to minimise the number of HTTP requests the ESP32 must handle.

4.2.1 PIN Gate Overlay

On page load the JavaScript calls /pin/status. If the response shows en: true and set: true, a full-screen PIN entry overlay is displayed, blocking access to the dashboard until the correct PIN is entered. The overlay calls /pin/login?p=XXXX; on success the server sets the g5t session cookie and the overlay is dismissed. Subsequent fetch() calls to /state and /cmd carry this cookie automatically.

4.2.2 Dashboard Tab

Thirteen sensor and control cards display live data. Cards glow green (active/OK), amber (warning), blue (fan/cooling), purple (security), or red (alarm/fire) using CSS box-shadow. The fire state machine produces a full-screen red flash overlay and fire banner when active. A Motion Detected banner appears at the top of the screen and auto-dismisses after 5 seconds.

4.2.3 System Security Tab

The System Security tab allows the user to set a new 4-digit PIN, change the existing PIN (requires entering the current PIN first), verify the current PIN, remove the PIN, and enable/disable PIN protection using a toggle switch. All changes call /pin/save on the server, which immediately persists the new configuration to hmi_pin.txt on flash. After three consecutive wrong PIN entries, the keypad locks for 30 seconds.

4.3 hmi_pin.txt — PIN Configuration File

This file is created automatically the first time a PIN is saved from the Security tab. Its format is a single line: pin,enabled — where pin is a 4-digit numeric string or empty (no PIN set), and enabled is 1 or 0. For example:

```
1234,1   (PIN 1234 set, protection enabled)
,0       (no PIN, protection disabled — factory default)
```

> Do not manually edit hmi_pin.txt while the ESP32 is running. Changes are overwritten on the next /pin/save call. To factory-reset security: delete hmi_pin.txt, or set its contents to ,0, and reboot.

5. Uploading Code to the ESP32

5.1 ESP32 NodeMCU-32S Reference

The ESP32 NodeMCU-32S is a 38-pin development board based on the Espressif ESP32-D0WDQ6 System-on-Chip. Key specifications:

Specification	Value
CPU	Xtensa LX6 dual-core, up to 240 MHz
Flash	4 MB SPI Flash (holds MicroPython + all project files)
SRAM	520 KB (shared between code heap and network stack)
WiFi	802.11 b/g/n 2.4 GHz, integrated antenna
GPIO	34 programmable I/O pins (some input-only)
ADC	2 × 12-bit SAR ADC, up to 18 channels
PWM	16-channel LEDC PWM (used for servo on GPIO18)
Operating voltage	3.3V (GPIO logic); VIN accepts 4.5–9V recommended via onboard LDO
USB interface	CP2102 or CH340 USB-to-UART bridge (varies by board supplier)
Board dimensions	Approximately 51 × 28 mm

The pinout diagram below shows the standard ESP32 NodeMCU-32S 38-pin layout with left and right rails:

LEFT PINS		ESP32 NodeMCU-32S (38 pins)		RIGHT PINS
3V3		— Top of board (USB end) —		GND
EN (RESET)				GPIO23 → Heater
VP / GPIO36 (input only)				GPIO22 → LED White
VN / GPIO39 (input only)				GPIO1 (TX0)
GPIO34 → Current ADC				GPIO3 (RX0)

GPIO35 → Fire sensor				GPIO21 → LED Blue
GPIO32 → LED Living Rm				GND
GPIO33 → Voltage ADC				GPIO19 → Buzzer
GPIO25 → LED Bedroom				GPIO18 → Servo PWM
GPIO26 → LED Bathroom				GPIO5 → Fan
GPIO27 → LED Kitchen				GPIO17
GPIO14 → DHT11				GPIO16
GPIO12				GPIO4 → Relay (Main Load)
GND				GPIO0 (Boot mode)
GPIO13 → PIR				GPIO2 → LED Red
SD2 (GPIO9)				GPIO15 → LED Green
SD3 (GPIO10)				SD1 (GPIO8)
CMD (GPIO11)		— Bottom (far from USB) —		CLK (GPIO6)
5V / VIN				SD0 (GPIO7)

Table: ESP32 NodeMCU-32S Pin Reference (project-used pins highlighted)

> GPIO6–GPIO11 are internally connected to the SPI flash and must not be used. GPIO34, GPIO35, GPIO36, GPIO39 are input-only — they have no internal pull-up or pull-down resistors.

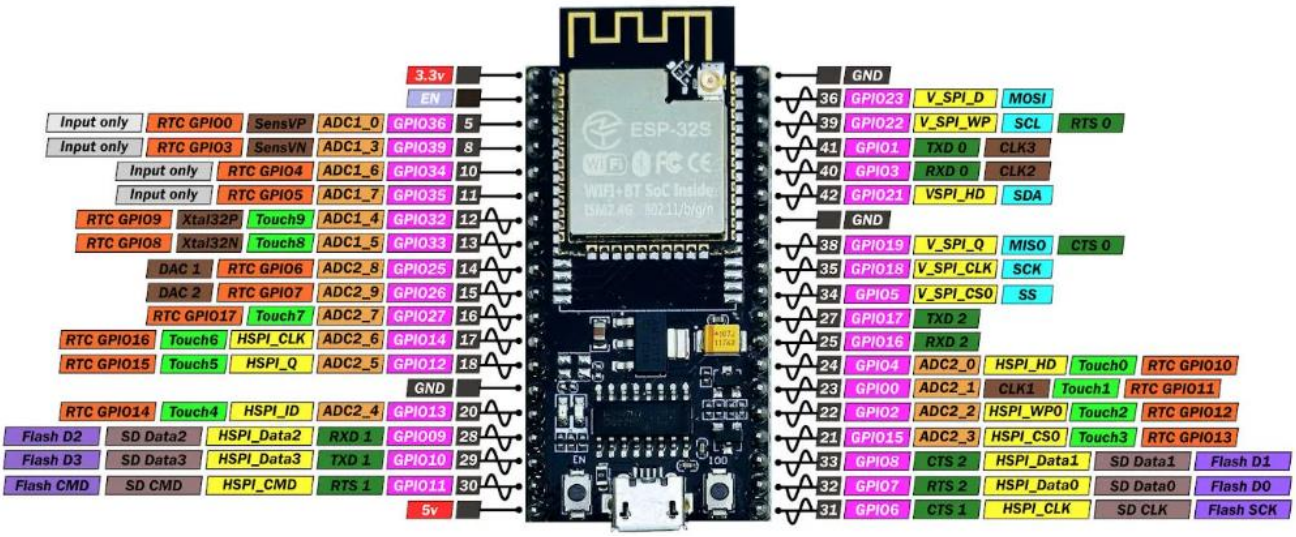

Fig. ESP32 NODEMCU-32S ESP-32S pinout

5.2 Installing MicroPython Firmware

If your ESP32 does not already have MicroPython installed, follow these steps:

1. Install Python 3 on your PC from python.org if not already installed.
2. Install esptool: open a terminal and run: pip install esptool
3. Download the latest MicroPython firmware for ESP32 from micropython.org/download/esp32/ — select the .bin file for the generic ESP32 board.
4. Hold the BOOT button on the ESP32 and plug it into your PC via USB.
5. Find the COM port (Windows Device Manager → Ports, or ls /dev/tty* on Linux/Mac).
6. Erase the existing flash (replace COMx with your port, e.g. COM5 or /dev/ttyUSB0):

```
esptool.py --port COMx erase_flash
```

7. Flash the MicroPython firmware (replace firmware.bin with your downloaded file name):

```
esptool.py --port COMx --baud 460800 write_flash -z 0x1000
firmware.bin
```

8. Release the BOOT button and press EN (reset). The ESP32 is now running MicroPython.

5.3 Uploading Project Files with Thonny

9. Download and install Thonny IDE from thonny.org.
10. Open Thonny. Go to Tools → Options → Interpreter.
11. Select MicroPython (ESP32) and choose the correct COM port. Click OK.
12. Plug the ESP32 into your PC via USB. Thonny should connect and show the MicroPython REPL at the bottom.
13. Open main.py in Thonny (File → Open).
14. Go to File → Save Copy As → MicroPython device. Name the file main.py and click OK. This uploads it to the ESP32 flash root.
15. Repeat for index.html — save it to the MicroPython device as index.html.
16. Create or upload hmi_pin.txt — either use File → New, type ,0, and save to device as hmi_pin.txt, or the firmware will create it with default values on first PIN save.
17. Press the EN button on the ESP32 or click the Stop/Restart button in Thonny to reboot and run main.py.
18. Watch the Thonny shell — you should see the boot sequence, WiFi connection, and IP address printed.
19. Open a browser and navigate to http://<IP_ADDRESS>. The G5 Smart Home HMI dashboard will load.

> After the three files are uploaded, the PC is no longer needed. The ESP32 can be powered from the 14V battery via the 5V buck converter and will boot and serve the HMI autonomously on every power cycle.

6. System Operation and Working

6.1 Boot Sequence

On power-up, MicroPython executes main.py automatically. The boot sequence is:

20. GPIO pins are configured and the green LED is lit (safe state).
21. A 2-second DHT11 warm-up delay allows the sensor to stabilise.
22. The WiFi station interface connects to the configured SSID. If connection fails within 12 seconds, an access point (G5-SmartHome, password: g5home123) is started as fallback.
23. hmi_pin.txt is read to load the saved PIN and enable/disable state.
24. The TCP server socket is bound to port 80 on all interfaces.
25. The main event loop begins: sensors poll every 1,000 ms; HTTP requests are served as they arrive.

6.2 Environmental Monitoring

The DHT11 sensor is polled every 3,000 ms (to respect its minimum sampling interval). Temperature is read in Celsius and converted to Fahrenheit by the firmware. Humidity is clamped to 0–100%. If three consecutive reads fail (checksum error or timeout), the dht_ok flag is cleared and the dashboard shows Sensor Err. The HVAC control logic only runs when dht_ok is true.

6.3 HVAC Automation

The heater (GPIO23) activates automatically when the measured temperature falls below the heat_setpoint (default 18°C). The fan (GPIO5) activates when temperature rises above cool_setpoint (default 28°C). Both setpoints are adjustable live from the dashboard without rebooting the ESP32. Changes take effect on the next sensor poll cycle.

6.4 Security and Motion Detection

The PIR sensor on GPIO13 is read every sensor cycle. On detecting motion (HIGH), the buzzer fires for 2 seconds, the motion flag is set, and a Motion Detected banner appears on the dashboard. The alarm auto-disarms after 10 seconds (ALARM_REARM_MS). A Disarm button on the dashboard silences the buzzer immediately and sets a 10-second re-arm timer. Motion is considered clear when the PIR has been LOW for 5 consecutive seconds.

6.5 Fire Detection

The fire sensor on GPIO35 is polled every sensor cycle. The state machine transitions are: when the sensor reads HIGH (fire), the system immediately enters the fire state, sounds no additional alarm (visual-only via LEDs and dashboard overlay), and lights the red LED. When the sensor returns LOW, a 5-second confirmation timer begins. After 5 seconds of clear readings, the system enters putoff state (blue LED, 20-second cooldown). After 20 seconds, it transitions to cooldown (one cycle) and then returns to safe with the green LED lit.

6.6 Power Measurement

The ADC reads are averaged over 8 samples to reduce noise. Voltage is computed as:

```
V = (adc_raw / 4095) × 3.6 × 11
```

Current is computed as:

```
I = (adc_raw / 4095) × 1.1 / 0.22
```

Power W = V × I. Energy accumulates only while the main load relay is active, incrementing in Wh based on elapsed milliseconds between polls. Cost in KSH is computed at 10 KSH/Wh. These accumulators never reset during a session (they reset on reboot).

6.7 Gate Control

The servo is driven with 50 Hz PWM via the hardware LEDC peripheral. The duty count maps as: 26 = 0° (gate closed), 123 = 180° (fully open). The gate is commanded to 90° (half-open) from the dashboard for normal operation. The gate state is reflected on the dashboard with a padlock icon and glow effect.

7. Challenges Faced and Solutions

7.1 HTTP Server — Truncated Requests (Bug 1)

The original recv loop read one 512-byte chunk and broke if it was smaller than 512 bytes, which happened on the very first receive for almost every request (browser GET headers with a Cookie header fit in a single TCP segment, typically 200–400 bytes). The handler received an incomplete or empty request and silently returned nothing, causing the browser to display a connection error.

Solution: The recv loop was replaced with a dedicated _recv_request() function that reads in 256-byte increments and exits only when the HTTP header terminator \r\n\r\n is found in the accumulated buffer, with a hard cap of 4 KB to prevent blocking on large POST bodies.

7.2 HTTP Server — Browser Timeout on Second Request (Bug 2)

The main loop accepted one client socket, processed it (serving the 49 KB HTML file could take 500–800 ms on a loaded ESP32), then returned to accept. Browsers open a second TCP connection almost immediately after receiving the HTML to fetch /pin/status (called by the JavaScript on page load). With a 0.5-second accept timeout and one connection handled per iteration, the second connection sat in the backlog until the server timed out and the browser gave up, resulting in a page that loaded but showed no live data.

Solution: The accept timeout was reduced to 50 ms and the main loop now drains up to four queued connections per iteration (inner for loop), ensuring that the HTML load and the immediately-following /pin/status request are both served before either times out.

7.3 DHT11 Sensor Read Failures

The DHT11 can occasionally produce checksum errors, particularly if polled faster than its 1-second minimum interval or if the data line has excessive capacitance. The firmware was updated to retry up to three times with an 80 ms delay between attempts, and to enforce a minimum 3,000 ms polling interval. The dht_ok flag prevents HVAC decisions being made on stale data.

7.4 ESP32 ADC Non-Linearity

The ESP32's built-in ADC is known for non-linearity at the extremes of its range (particularly near 0V and near full scale). For this project, voltage and current readings are averaged over 8 samples to reduce noise, and the operating range is kept within the linear middle portion of the ADC range by design (the 10kΩ/1kΩ divider keeps the voltage reading well within 0.5–3.0V at 14V supply).

7.5 Power Bank / Battery Auto-Shutoff

Some power banks cut power when the load current drops below a threshold (typically 50–100 mA). The ESP32 with WiFi active draws approximately 80–150 mA, which is usually sufficient to keep a power bank alive. For this project a 14V battery is used with a buck converter, which does not have auto-shutoff behavior, making it more reliable for continuous 24/7 operation.

7.6 MicroPython Heap Memory

The ESP32 has 520 KB SRAM shared between the MicroPython runtime, the WiFi stack, and application code. Loading a 49 KB HTML file into RAM would exhaust the heap. The _stream_html() function was implemented to read and send the file in 1 KB chunks directly from flash, never loading it fully into RAM. gc.collect() is called after each request to reclaim heap fragments from JSON serialisation.

8. GitHub README Summary

> The following section is the content of the README.md file to be placed in the GitHub repository root.

```
# G5 Smart Home HMI
### ESP32 NodeMCU-32S Residential Automation Controller
```

A self-contained smart home controller running MicroPython on the ESP32 NodeMCU-32S. The device hosts its own web server and serves a full-featured browser dashboard from internal flash. No cloud, no app, no internet required — just connect to the same WiFi network and open a browser.

```
## Features
- Real-time temperature and humidity (DHT11)
- Motion detection alarm with auto-rearm (PIR HC-SR501)
- Fire detection state machine with RGB LED indicators
- Servo-controlled gate with HMI open/close buttons
- Four-room LED lighting control
- Automatic HVAC: heater and fan controlled by setpoints
- Main load relay (fridge / appliance)
- White security light toggle
- Power monitoring: voltage, current, power, energy (Wh), cost (KSH)
- 4-digit PIN security — persisted to flash, survives reboot
- Vibrant colour-coded dashboard with fire overlay and motion banner

## Files
- main.py — MicroPython firmware (sensor polling, HTTP server, PIN auth)
- index.html — Single-page dashboard (HTML/CSS/JS, served from ESP32
  flash)
- hmi_pin.txt — PIN config (auto-generated; format: pin,enabled e.g.
  1234,1)

## Hardware Requirements
- ESP32 NodeMCU-32S
- DHT11 temperature/humidity sensor
- PIR motion sensor (HC-SR501)
- Flame/fire sensor module
- SG90/MG90S servo motor
- 5V single-channel relay module
- Passive buzzer
- LEDs: Red (GPIO2), Green (GPIO15), Blue (GPIO21), White (GPIO22), 4x
  room LEDs
```

- 12V incandescent bulbs (300 mA) for heater and main load
- 12V DC fan
- 14V DC battery + 5V buck converter
- 0.22Ω shunt resistor and 10kΩ/1kΩ voltage divider for ADC
- NPN transistors, resistors, capacitors, perfboard

Quick Start

- 1. Flash MicroPython firmware to ESP32 using esptool.py
- 2. Open Thonny IDE, connect to ESP32 (MicroPython interpreter)
- 3. Upload main.py, index.html, and hmi_pin.txt to the ESP32 flash root
- 4. Set your WiFi SSID and password in main.py (SSID and PASSWORD constants)
- 5. Press EN (reset) on the ESP32 — watch the Thonny shell for the IP address
- 6. Open a browser and navigate to http://<ESP32_IP>
- 7. Once uploaded, the PC is no longer needed — power from battery

PIN Security

Navigate to the System Security tab in the dashboard. Use the keypad to set a 4-digit PIN. Toggle the switch to enable. When enabled, any browser visiting the IP must enter the PIN before accessing the dashboard. The PIN survives reboots (stored in hmi_pin.txt on flash). Factory reset: set hmi_pin.txt contents to ,0

License

Business Source License 1.1 (BUSL-1.1) — source available, non-production use permitted. Production use requires a commercial licence from the author. See LICENSE file for full terms. Copyright © 2026 Abraham Winston Ogoch.

Appendix

Appendix A: Circuit Diagram

The full circuit schematic will be inserted here. The diagram should show all GPIO connections, transistor switching circuits, the 14V battery supply rail, 5V buck converter output, voltage divider for GPIO33, shunt resistor for GPIO34, relay module wiring, servo connection, and all LED series resistors.

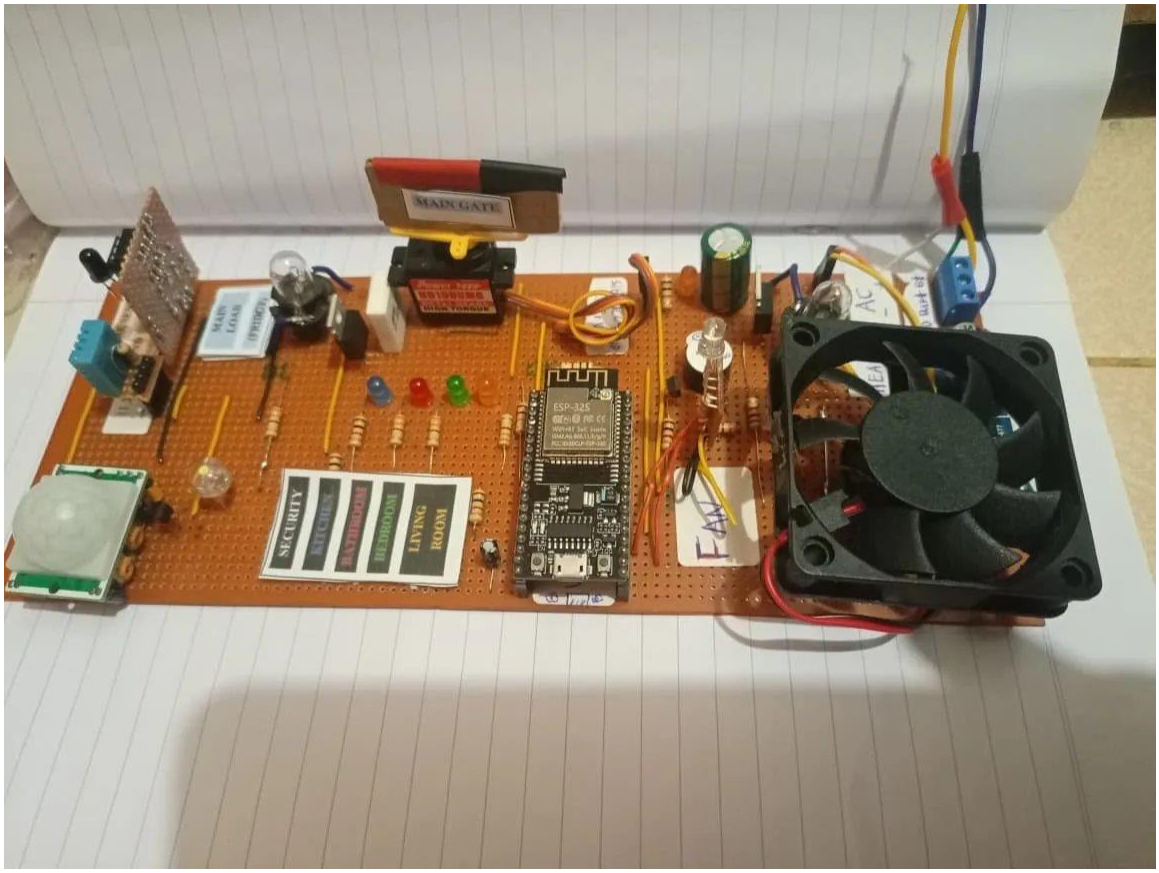

Figure A1: G5 Smart Home HMI — Top View (overhead)

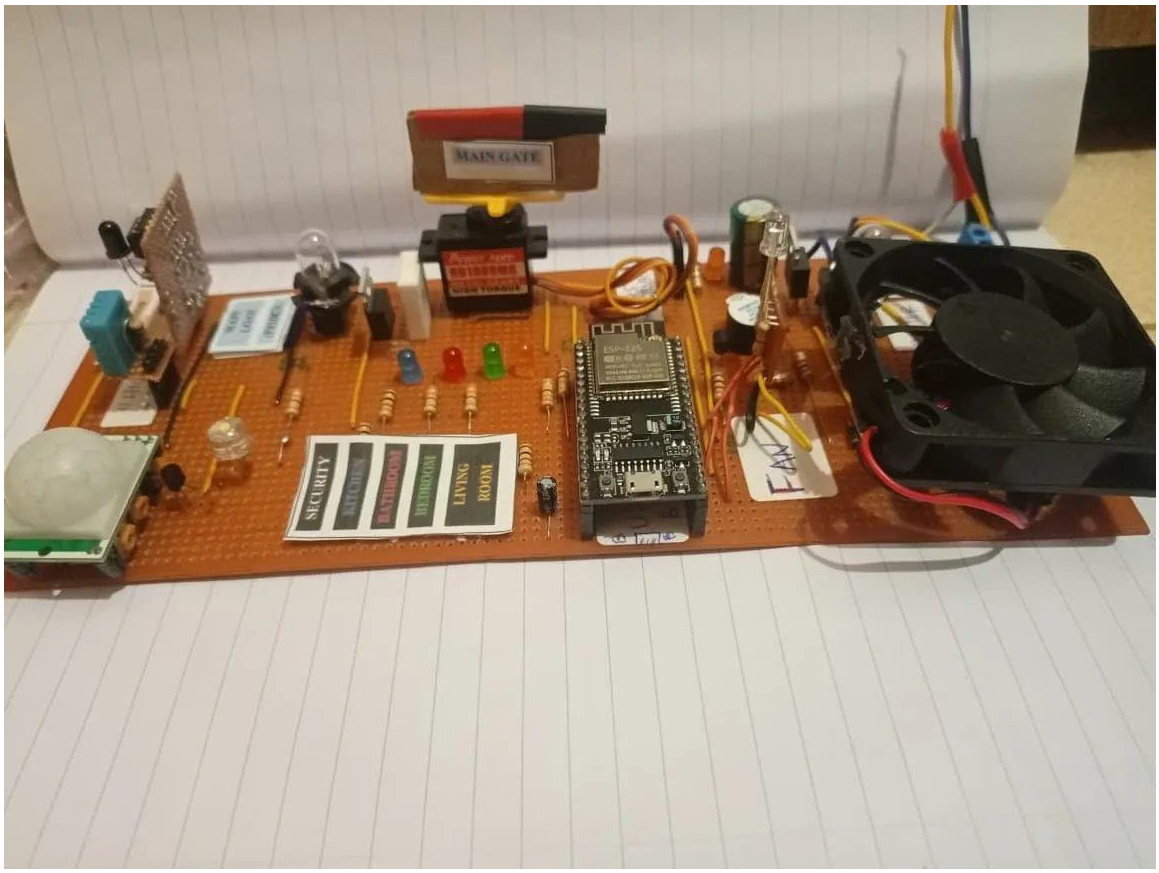

Figure A2: G5 Smart Home HMI — Side View

Appendix B: Firmware Configuration Constants

Constant	Default Value	Description
SSID	"Winstonxc"	WiFi network name to connect to (Mobile Hotspot)
PASSWORD	"nilexd254"	WiFi password (Mobile Hotspot Password--- **not sensitive**)
COST_PER_WH	10.0	Energy cost in KSH per Wh
ALARM_REARM_MS	10,000 ms	Alarm re-arm delay after manual disarm
FIRE_PUTOFF_MS	5,000 ms	Confirmation delay before entering putoff state
BLUE_HOLD_MS	20,000 ms	Duration of blue LED cooldown after fire clears
HTML_FILE	"index.html"	Filename of the dashboard served from flash
PIN_FILE	"hmi_pin.txt"	Filename of the PIN configuration file

heat_setpoint (default)	18.0 °C	Temperature below which heater activates (adjustable from HMI)
cool_setpoint (default)	28.0 °C	Temperature above which fan activates (adjustable from HMI)

Appendix C: API Endpoint Reference

Endpoint	Method	Auth?	Description
/	GET	Yes	Serves index.html from flash (streams 1 KB chunks)
/state	GET	Yes	Returns full state dictionary as JSON
/cmd	GET	Yes	Accepts query params: gate, room, val, load, heat_sp, cool_sp, disarm, white. Returns updated state JSON.
/pin/status	GET	No	Returns {set: bool, en: bool}. Always accessible — used by browser to decide whether to show PIN overlay.
/pin/login?p=XXXX	GET	No	Validates PIN. On success: sets g5t session cookie, returns {ok: true}. On failure: {ok: false}.
/pin/save	GET	Yes	Params: p (new PIN or empty), en (1/0). Saves to hmi_pin.txt immediately.

Appendix D: Troubleshooting Guide

Symptom	Likely Cause and Solution
Browser shows 'site can't be reached'	ESP32 not on same WiFi network, or wrong IP. Check Thonny shell for printed IP address. Try pinging the IP from your PC.
Page loads but shows no data	Bug 2 (second connection timeout). Ensure main.py uses the updated server loop with srv.settimeout(0.05) and the 4-connection drain loop.
DHT11 shows Sensor Err	Check wiring: 4.7kΩ pull-up to 3.3V on data pin. Ensure VCC is 3.3V (not 5V for 3.3V-only DHT11 modules). Allow 2 s warm-up.
Gate servo jitters or doesn't move	Insufficient 5V current for servo. Use a dedicated 5V supply (not the ESP32 3V3 pin). Check servo PWM frequency is 50 Hz.
ADC voltage reads ~0 or garbage	Check 10kΩ/1kΩ divider wiring to GPIO33. Confirm ATTN_11DB is set. Ensure divider GND is the same as ESP32 GND.

PIN gate appears but correct PIN rejected	Check that hmi_pin.txt contains the correct PIN. Use Thonny file manager to inspect the file. Session cookie may have expired (1-hour TTL).
ESP32 keeps rebooting	Brownout from insufficient supply current. Ensure the 5V buck converter is rated ≥1A. Check USB cable quality if uploading from PC.
Fire sensor triggers constantly	Sensor too close to bright light or sunlight. Adjust the potentiometer on the sensor module to reduce sensitivity.

Appendix E: Dashboard and Software Screenshots

E.1 Main Dashboard — Normal Operation

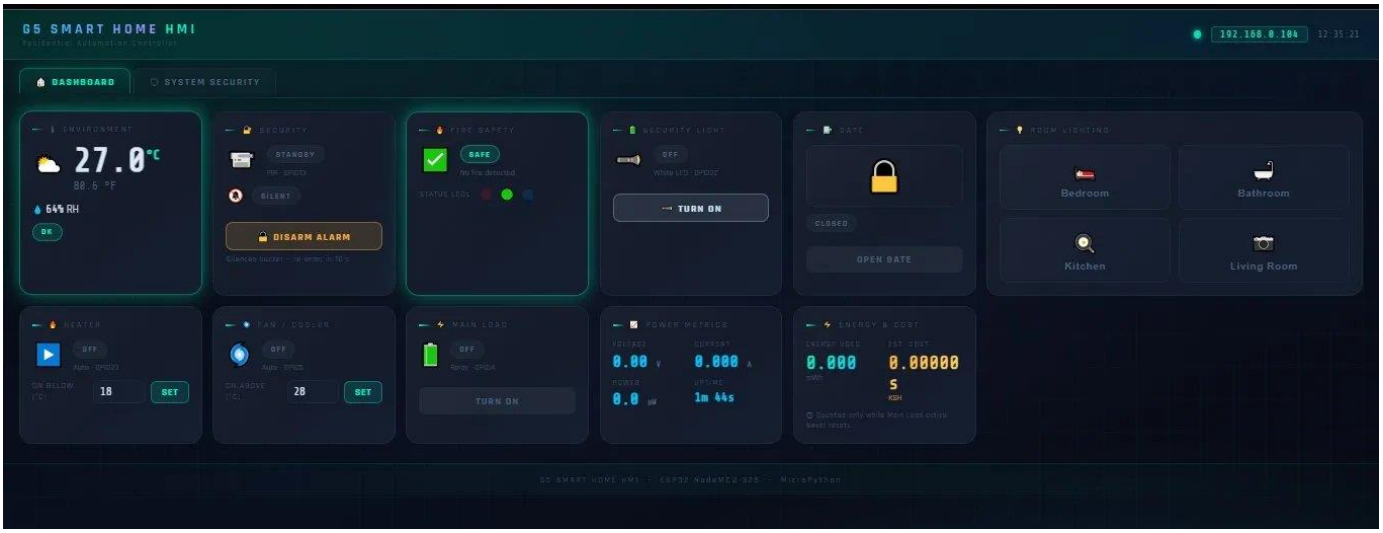

Figure E1: Main Dashboard — all systems nominal, 27.0 °C, 64% RH, gate closed

E.2 Dashboard — Motion and Fire Alarm Active

Figure E2: Dashboard with simultaneous Motion Detected and Fire Detected alerts active

E.3 Dashboard — Fire Cooldown State

Figure E3: Fire put out — 20-second blue LED cooldown timer active; gate open, heater and fan running

E.4 PIN Entry Overlay

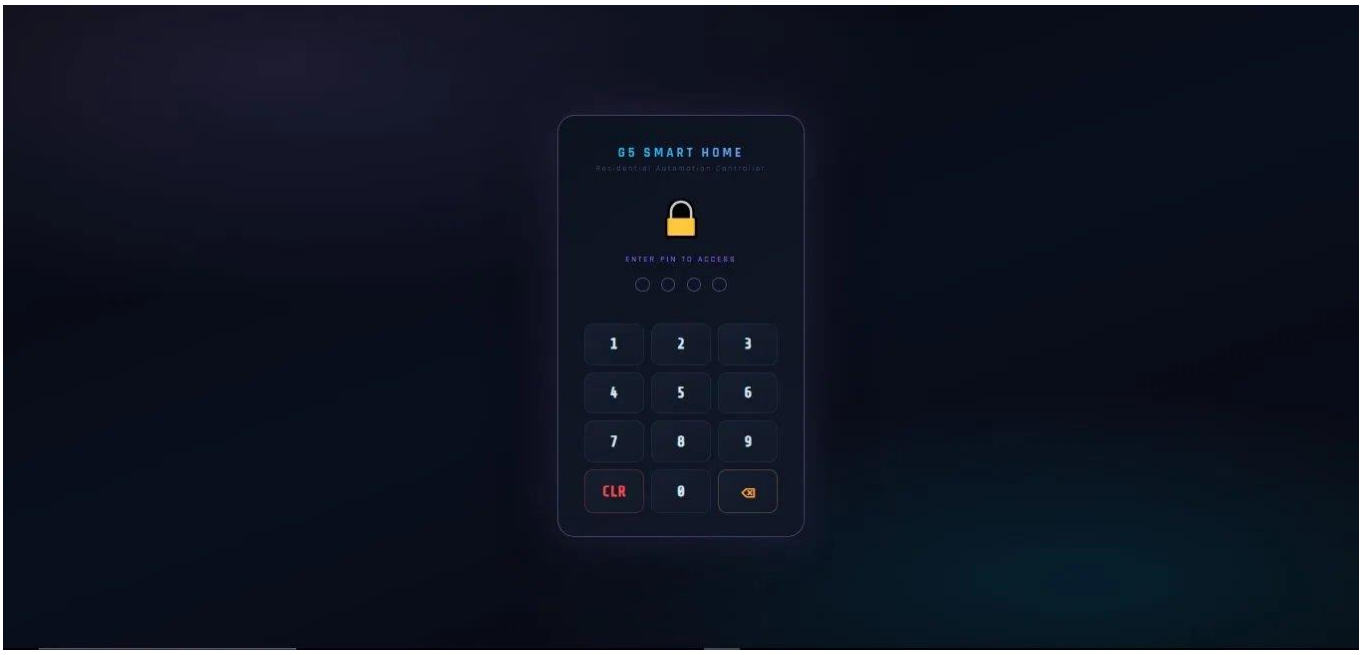

Figure E4: Full-screen PIN entry overlay displayed before dashboard access when PIN protection is enabled

E.5 System Security Tab

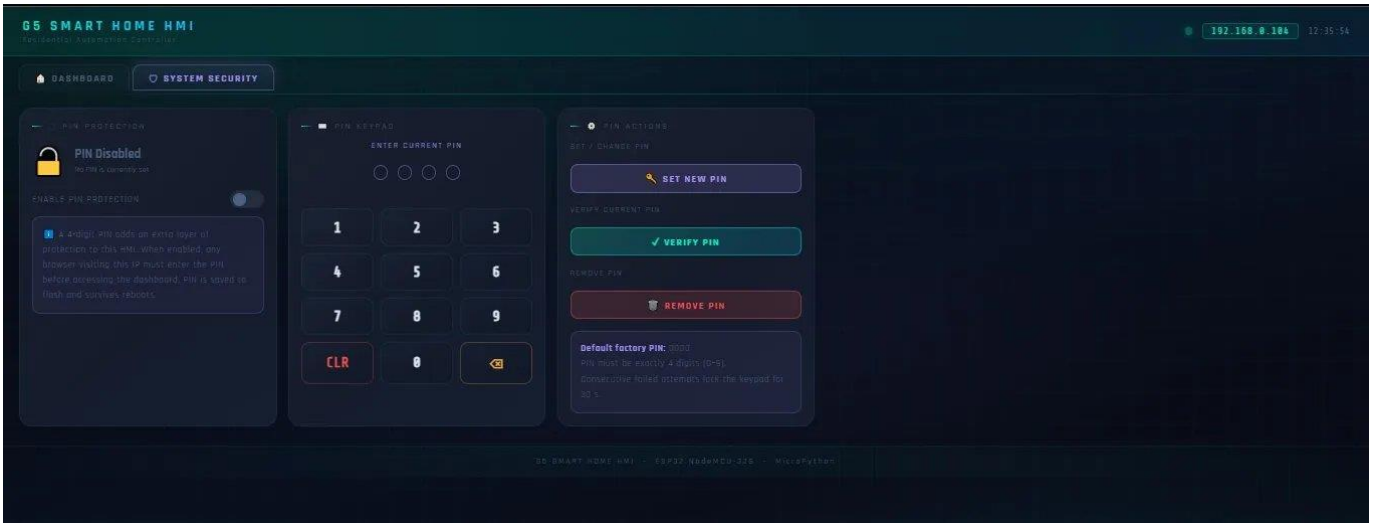

Figure E5: System Security tab — PIN keypad, Set/Verify/Remove PIN actions, enable toggle

E.6 Thonny IDE — Firmware Upload

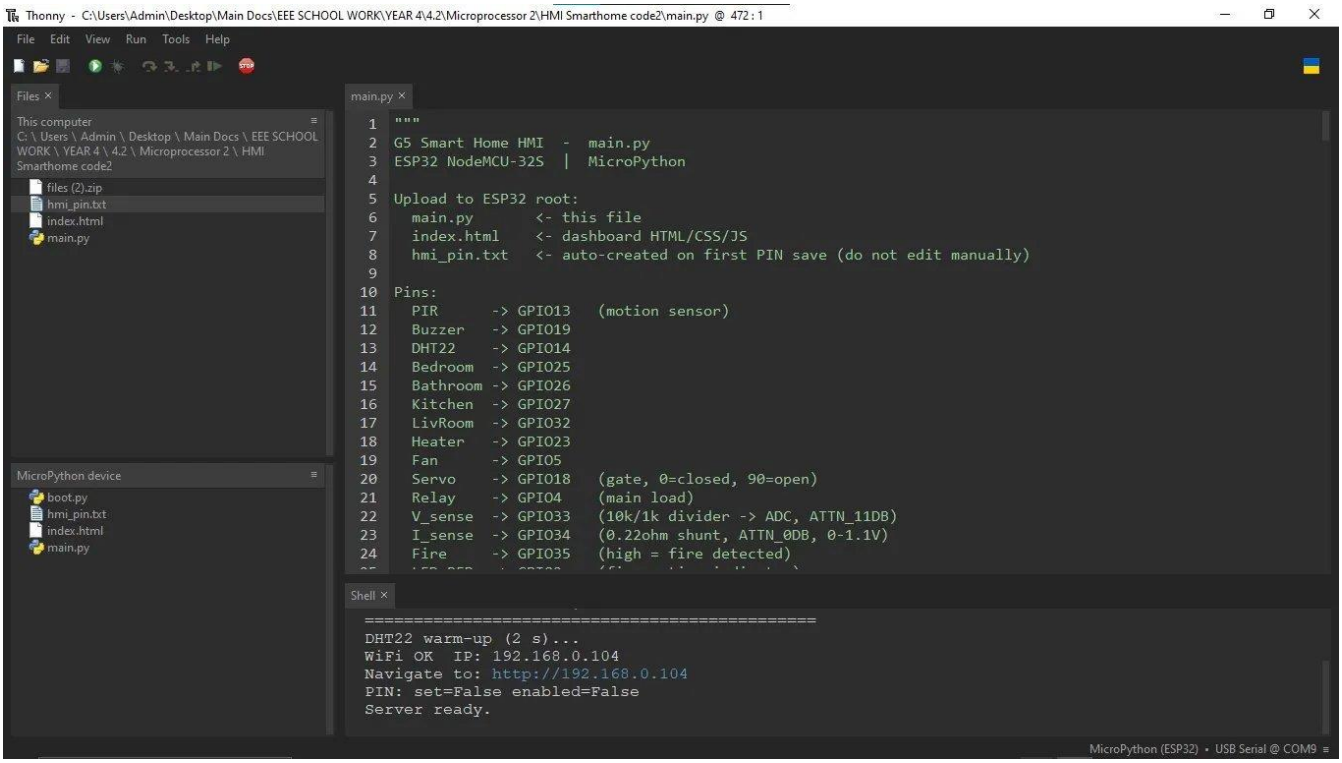

Figure E6: Thonny IDE showing main.py open with pin mapping header and MicroPython device files

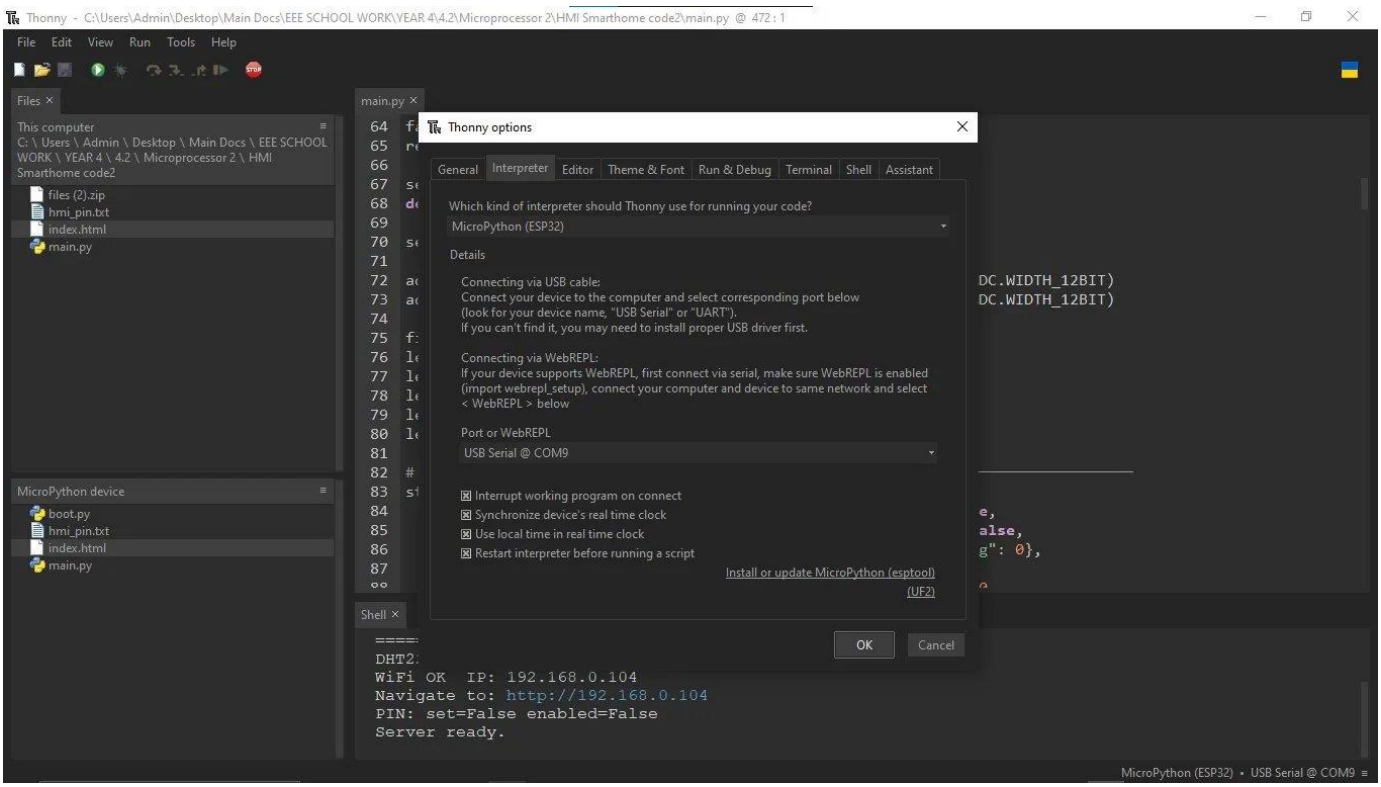

Figure E7: Thonny Interpreter options — MicroPython (ESP32) selected on USB Serial @ COM9

— End of Document —